

03 — High-Level Design (HLD)

1. System context

```
flowchart LR
    user([Customer browser])
    admin([Admin browser])
    swa[Azure Static Web Apps<br/>React SPA]
    aca[Azure Container Apps<br/>NestJS API /api]
    pg[(Azure PostgreSQL<br/>Flexible Server)]
    blob[(Azure Blob<br/>media container)]
    ai[Azure App Insights]
    acr[(Azure Container Registry)]
    smtp[SMTP / SendGrid]
    stripe[Stripe]
    tabby[Tabby]
    tamara[Tamara]

    user --> swa
    admin --> swa
    swa -- HTTPS --> aca
    aca --> pg
    aca --> blob
    aca --> smtp
    aca --> stripe
    aca --> tabby
    aca --> tamara
    stripe -- webhook --> aca
    tabby -- webhook --> aca
    tamara -- webhook --> aca
    aca -- traces/logs --> ai
    acr -- pull image --> aca
```

2. Component view

```

flowchart TB
  subgraph Frontend [React SPA - Vite, TS]
    routes[React-Router routes]
    stores[Zustand stores<br/>auth · cart · favorites · home]
    apiclient[fetch client + auth interceptor]
    pages[Pages]
    components[Reusable components]
    routes --> pages --> components
    pages --> stores --> apiclient
  end

  subgraph Backend [NestJS API]
    main[main.ts<br/>helmet · cors · validation · throttler]
    auth[Auth module<br/>JWT · refresh · password]
    catalog[Catalog · Products · Categories · Brands · Designers]
    cms[CMS · Banners · Home · Navigation · Landing · Collections · Announcements]
    cart[Cart]
    orders[Orders · Returns · Invoices]
    payments[Stripe · Tabby · Tamara]
    admin[Admin · Reports · Stock · Customers · Settings]
    cross[Common · Throttler · Guards · Interceptors · Pino logger · App Insights]
    main --> auth & catalog & cms & cart & orders & payments & admin
    auth & catalog & cms & cart & orders & payments & admin --> cross
  end

  subgraph Data [PostgreSQL]
    schema[(Prisma schema<br/>~50 tables, enums, indexes)]
  end

  Frontend -- /api --> Backend
  Backend -- Prisma --> Data

```

3. Tech stack summary

Layer	Choice	Rationale
Frontend framework	React 19 + Vite 6 + TS 5	Fast HMR, modern SSG-ready, small bundles
State	Zustand 5	Tiny, ergonomic, no boilerplate
Routing	react-router-dom 7	Standard SPA routing, lazy loading
Styling	CSS modules + design tokens	Predictable, no CSS-in-JS overhead
Backend framework	NestJS 10	DI + module boundaries + good test ergonomics
ORM	Prisma 5	Type-safe DB layer, migrations

Layer	Choice	Rationale
DB	PostgreSQL 14	Mature, JSON support, full-text search ready
Auth	Passport-JWT + Argon2id	Industry-standard hashing, stateless tokens
Validation	class-validator + Joi (env)	DTO-level + boot-time validation
Logging	nestjs-pino	Structured JSON logs, fast
Telemetry	Application Insights SDK	Native Azure observability
Caching	@nestjs/cache-manager	Optional in-memory cache for hot reads
Throttling	@nestjs/throttler	Per-route rate limits
Files / images	sharp + Azure Blob	On-the-fly resize, durable storage
PDF	pdfkit	Invoice rendering
Email	nodemailer	Transactional email via SMTP
Payments	Stripe SDK + custom Tabby/Tamara	UAE-friendly mix

4. Deployment topology (Azure)

flowchart LR

```

subgraph rg [Resource Group: rg-Solo-Website]
  direction TB
  swa[Static Web App<br/>frontend]
  env[Container Apps Env]
  aca[Container App<br/>backend]
  acr[ACR]
  mi[User-assigned MI<br/>AcrPull]
  pg[(PostgreSQL Flexible)]
  sa[Storage Account<br/>media container]
  log[Log Analytics WS]
  ai[App Insights]
  env --- aca
  aca -- pull --> acr
  aca -- uses --> mi
  aca -- TCP 5432 --> pg
  aca -- HTTPS --> sa
  aca -- traces --> ai
  ai --- log
end
swa -- /api proxy --> aca

```

Provisioned by [infra/main.bicep](#) and orchestrated by `azd up` / `azd deploy`. See [09 Deployment & Operations](#) for the runbook.

5. Request lifecycle

```
sequenceDiagram
    participant C as Client
    participant SWA as Static Web App (CDN)
    participant ACA as Container App
    participant Nest as Nest pipeline
    participant Svc as Service
    participant Pr as Prisma
    participant PG as PostgreSQL
    C->>SWA: GET /products/123 (HTML)
    SWA-->>C: SPA shell
    C->>ACA: GET /api/products/123 (JWT?)
    ACA->>Nest: HTTPS
    Nest->>Nest: Helmet, CORS, body-parse, ValidationPipe, Throttler
    Nest->>Nest: AuthGuard (optional) + RolesGuard
    Nest->>Svc: ProductsService.findOne
    Svc->>Pr: prisma.product.findUnique
    Pr->>PG: SELECT
    PG-->>Pr: row
    Pr-->>Svc: typed result
    Svc-->>Nest: DTO
    Nest-->>ACA: JSON
    ACA-->>C: 200
```

6. Cross-cutting concerns

Concern	Implementation
Authentication	<code>JwtAuthGuard</code> (Passport-JWT). Tokens signed HS256, 15-min access TTL, refresh tokens persisted & rotated
Authorisation	<code>RolesGuard</code> reads <code>@Roles(...)</code> decorator metadata; enum <code>UserRole</code>
Validation	Global <code>ValidationPipe({ whitelist: true, transform: true })</code>
Errors	Global <code>HttpExceptionFilter</code> returns <code>{ statusCode, message, error, timestamp, path }</code>
Logging	nestjs-pino HTTP middleware + child loggers per service
Tracing	<code>tracing.ts</code> boots Application Insights before Nest
Rate limiting	Global <code>ThrottlerGuard</code> ; <code>/auth/*</code> overridden to 5 / 15 min

Concern	Implementation
Security headers	Helmet defaults
CORS	Allowlist of SWA origins from <code>CORS_ORIGINS</code> env
Caching	Per-controller <code>CacheInterceptor</code> on hot read paths
Soft delete	<code>deletedAt</code> filters in product queries
Idempotency	<code>stripe_events.id</code> unique-key dedup; refresh token rotation
Migrations	Prisma migrations in <code>backend/prisma/migrations</code>

7. Data flow — checkout (PAID happy path)

```
sequenceDiagram
    participant SPA
    participant API
    participant DB
    participant Stripe
    SPA->>API: POST /api/orders {addressId, shippingMethod, paymentMethod, items, promoC
    API->>DB: BEGIN; lock product rows; compute totals; create Order(PAYMENT_PENDING); c
    DB-->>API: orderId
    API-->>SPA: {orderId, total}
    SPA->>API: POST /api/stripe/create-payment-intent {orderId}
    API->>Stripe: PaymentIntent.create(amount, currency=AED, metadata={orderId})
    Stripe-->>API: {clientSecret, id}
    API-->>SPA: {clientSecret}
    SPA->>Stripe: confirmCardPayment(clientSecret)
    Stripe-->>SPA: succeeded
    Stripe->>API: POST /api/stripe/webhook (signed)
    API->>DB: UPSERT stripe_events; if new => order.status=PAID, decrement stock, write
    API-->>Stripe: 200
    SPA->>API: GET /api/orders/:id (poll until PAID)
```

8. Scalability strategy

- ACA replicas: scale by HTTP concurrency (default min 1, max 10).
- Postgres: vertical scale (Flexible Server SKU); read replicas planned.
- Cache hot category/brand/home reads in-memory (process-level) with short TTLs; future: Redis cache when multi-replica.
- Static assets (images) served from Blob via SWA-managed routes; can be fronted by Azure CDN/Front Door.

9. Failure modes & resilience

Scenario	Mitigation
DB connection blip	Prisma reconnect; opossum circuit breaker around 3rd-party calls
Stripe webhook lost	Stripe auto-retries with exp backoff; idempotent on our side
Out-of-stock race	Decrement inside a transaction; return 409 if reservation fails
Bad JWT / expired	401 with hint to refresh; refresh endpoint returns new pair
Image upload failure	Multipart upload returns 4xx; UI surfaces the error
Email outage	Nodemailer retries; non-fatal — DB state remains consistent

10. Future evolution

- Read replicas for catalog
- Redis cache for cart sessions and rate-limit counters
- Multi-currency display
- AR locale UI
- Native mobile app reusing the same `/api`
- ETL → BI pipeline (Azure Synapse / Fabric)